Gain Attribution with Floor Check

Leak-free predictive training with a shared encoder, SIGReg and slow-feature protection. **Rule:** the encoder may simplify the target latent only in directions where the predictor has stopped improving *and* sits near its achievable error floor.

1 Problem

SIGReg checks only batch statistics. Slow features vary strongly between clips, so they satisfy it while being trivially predictable, and the encoder can collapse onto them. Stop-grad on the target does **not** prevent this: the encoder is shared, so a context-path update $\Delta\theta$ also moves $z_t = f(x_t; \theta)$. If that makes z_t easier to predict, the loss drops through a route no gradient accounted for (the **leak**).

2 Notation

$z_c = f(x_c; \theta)$, $z_t = f(x_t; \theta)$; $p = P(z_c)$ predictor output; P = lagged predictor (EMA copy of P). j indexes latent directions (dims or PCA axes). L_j = batch-mean $(p_j - z_{t,j})^2$, $L = \Sigma_j L_j$. S = set of directions where the predictor is still learning (defined in 4).

$\mathbf{g_{ctx}}$	**Context-path gradient.** $\nabla_\theta L$ with z_t stop-gradded. What autograd gives; normally the only encoder gradient.
$\mathbf{g_{tgt,S}}$	**Restricted target-path gradient.** $\nabla_\theta \Sigma_{j\in S} L_j$ through z_t, p detached. Hypothetical: used only as a measuring stick, never applied.
$\mathbf{g'_{ctx}}$	**Projected context gradient.** g_{ctx} with the leak component removed. This is what gets applied.
$\mathbf{g_{open}}$	**Open-target gradient.** $\nabla_\theta \Sigma_{j\notin S} L_j$ through z_t: the encoder helps where the predictor is saturated.
$\mathbf{g_{SIG}}$	**SIGReg gradient.** Anti-collapse term, never projected.

3 Floor estimate L^*_j (new)

Every N steps: freeze a snapshot of the *current* encoder, take its latents on a held-out pool, train a fresh probe predictor (same architecture as P) for budget B, and record the held-out per-direction error. L^*_j is the best error reachable in direction j given today's latents, including irreducible noise. Smooth across refreshes (EMA). Use snapshots from the real run, not a separate frozen run, so the drift you want to catch is present.

$$e_j = (L_j - L^*_j) / L^*_j \text{ (excess over floor; } \approx 0 \text{ means at the floor)}$$

4 Progress gate and learning set S

Lagged-predictor gap, both predictors evaluated on the **same batch and latents** (isolates predictor progress from latent drift), averaged over window W:

$$\text{gap}_j = \text{EMA}_W[\, L_j(P) - L_j(P) \,]$$

$$j \in S \Leftrightarrow \text{gap}_j > \varepsilon_g \text{ or } e_j > \varepsilon_f$$

$\mathbf{gap_j}$	$\mathbf{e_j}$	State	Action
high	high	still learning	in S: protect
≈ 0	high	**plateau** (stuck, not done)	in S: protect (floor check catches this)
high	≈ 0	nearly done	in S until gap decays
≈ 0	≈ 0	**saturated**: solved or noise floor	not in S: encoder may help

Use hysteresis: enter S at the threshold, leave only at half of it, plus a minimum dwell time, so directions do not flicker.

5 Target path: open only where saturated

$$L_{open} = \Sigma_{j\notin S} L_j(\, p.\text{detach}(), z_t \,) \rightarrow g_{open} = \nabla_\theta L_{open}$$

Optional soft version: weight $w_j = 1 - \text{clip}(\text{gap}_j / \text{gap}_{ref}, 0, 1)$ per direction.

6 Context path: leak measurement and projection

To first order, a context step $\Delta\theta = -\eta \cdot g_{ctx}$ changes the loss by

$$\Delta L \approx -\eta \|g_{ctx}\|^2 - \eta\, (g_{tgt,S} \cdot g_{ctx})$$

The first term is legitimate learning through the context. The second is the **leak**: loss reduction obtained by moving the target in still-learning directions. Define $\ell = g_{tgt,S} \cdot g_{ctx}$. If $\ell > 0$, remove that component:

$$g'_{ctx} = g_{ctx} - \alpha \cdot \max(0, \ell) / \|g_{tgt,S}\|^2 \cdot g_{tgt,S}$$

With $\alpha = 1$, $g'_{ctx} \cdot g_{tgt,S} = 0$, so the context step can no longer lower the loss via the target on S. On saturated directions nothing is projected. Both paths then obey one rule: in learning directions only the predictor reduces the loss; in saturated directions the encoder may help.

7 Full update

$$\theta \leftarrow \theta - \eta \cdot \text{Opt}(\, g'_{ctx} + g_{open} + \lambda\, g_{SIG} \,)$$

Predictor P: standard step on L. Lagged predictor: $P \leftarrow \text{EMA}(P)$. Floor L^*_j: refreshed every N steps.

8 Training step (sketch)

```
z_c, z_t = f(x_c), f(x_t)            # keep z_t graph
p = P(z_c);  Lj = per_dir_mse(p, z_t.detach())
gap = ema(Lj_lag - Lj);  e = (Lj - Lstar)/Lstar
S = hysteresis(gap > eps_g  or  e > eps_f)
g_ctx  = grad(Lj.sum(), theta, retain_graph=True)
g_tgtS = grad(per_dir_mse(p.detach(), z_t)[S].sum(), theta,
              retain_graph=True)
g_open = grad(per_dir_mse(p.detach(), z_t)[~S].sum(), theta)
l = dot(g_tgtS, g_ctx)
if l > 0: g_ctx -= alpha*l/dot(g_tgtS,g_tgtS) * g_tgtS
step(g_ctx + g_open + lam*g_sigreg)
every N steps: Lstar = train_probe(frozen_copy(f))
```

9 Diagnostics

Per direction: gap_j, e_j, L^*_j, share of steps with $\ell > 0$, $\|g'_{ctx}\|/\|g_{ctx}\|$. Cheap per-direction leak via one forward-mode JVP: $\Delta z_t = \text{JVP of } f \text{ at } x_t$ along $-\eta \cdot g_{ctx}$, then $\text{leak}_j = -\text{mean}(\partial L_j/\partial z_{t,j} \cdot \Delta z_{t,j})$. **Exact check every N steps:** $\ell_{real} = L(\theta', z_t(\theta)) - L(\theta', z_t(\theta'))$; it should be ≈ 0 on S after projection. Also track the within-/between-clip variance ratio of z and the effective rank of Δz (slow-feature drift). Comparing leak with vs. without an EMA target explains why EMA helps.

10 Cost and caveats

Cost: +1 lagged-predictor forward; +1 backward through the target path (target graph kept, about 2× encoder activations; cut by using the last few layers or a sub-batch); probe training amortized every N steps.

- **Probe bias:** an under-trained probe overestimates L^*_j, so e_j looks small and the gate opens too early. Use a generous budget, a margin, and keep the min over recent refreshes.
- **Stale floor:** L^*_j drifts as latents move; refresh regularly.
- **Over-projection:** g_{ctx} and g_{tgt} are correlated via shared weights. If training slows, lower α or project only above a threshold on ℓ.
- **Optimizer:** derivation assumes plain SGD. With Adam, use the real preconditioned update direction or verify with ℓ_{real}.
- **First order only:** large η weakens the leak estimate.

Hyperparameters

W	gap window	$\boldsymbol{\varepsilon_g}$, $\boldsymbol{\varepsilon_f}$	gap / excess thresholds
N, B	floor refresh, probe budget	$\boldsymbol{\alpha}$	projection strength (1 = full)
$\boldsymbol{\tau}$	EMA constant of P	$\boldsymbol{\lambda}$	SIGReg weight